

JRadioButton es el botón visible. ButtonGroup es el objeto que hace que los botones del mismo grupo sean mutuamente excluyentes. El ButtonGroup no aparece en la ventana.

2.1. Crear los componentes en NetBeans

- 1 Crea un proyecto Java con interfaz Swing.
- 2 Abre tu JFrame Form.
- 3 Desde Palette, arrastra tres componentes Radio Button.
- 4 Cambia sus textos a Mañana, Tarde y Noche.
- 5 Cambia sus nombres de variable a rbtnManana, rbtnTarde y rbtnNoche.
- 6 Usa un Button Group y asigna los tres radios al mismo grupo.

2.2. Saber cuál está seleccionado

El método isSelected() devuelve true si el radio está seleccionado y false si no.

```
if (rbtnManana.isSelected()) {  
    System.out.println("Seleccionó mañana");  
} else if (rbtnTarde.isSelected()) {  
    System.out.println("Seleccionó tarde");  
} else if (rbtnNoche.isSelected()) {  
    System.out.println("Seleccionó noche");  
} else {  
    System.out.println("No seleccionó una jornada");  
}
```

2.3. Ejemplo con un botón

Haz doble clic sobre un JButton, por ejemplo btnAceptar, para generar su actionPerformed.

```
private void btnAceptarActionPerformed(java.awt.event.ActionEvent evt) {  
    String jornada;  
  
    if (rbtnManana.isSelected()) {  
        jornada = "Mañana";  
    } else if (rbtnTarde.isSelected()) {  
        jornada = "Tarde";  
    } else if (rbtnNoche.isSelected()) {  
        jornada = "Noche";  
    } else {  
        jornada = "No seleccionada";  
    }  
  
    JOptionPane.showMessageDialog(  
        this, "Jornada elegida: " + jornada  
    );  
}
```

2.4. Crear el grupo mediante código

```
ButtonGroup grupoJornada = new ButtonGroup();
```

```
grupoJornada.add(rbtnManana);  
grupoJornada.add(rbtnTarde);  
grupoJornada.add(rbtnNoche);
```

Comprender este código te ayuda a entender lo que conceptualmente hace NetBeans cuando configura el grupo.

2.5. Error frecuente

Para saber si un JRadioButton está marcado, usa isSelected(). No necesitas comparar manualmente el texto del botón.

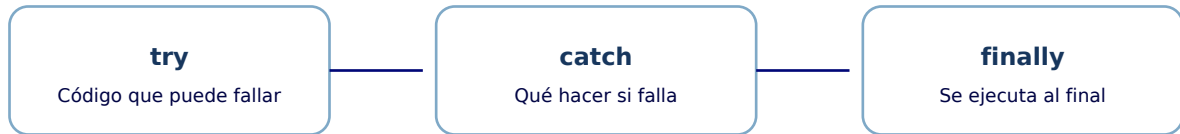
2.6. Mini-práctica

Crea un formulario con Estudiante, Docente y Administrativo. Agrúpalos y muestra un mensaje según la opción seleccionada.

3. Manejo de excepciones

¿Qué es una excepción?

Es una situación anormal que ocurre mientras el programa se ejecuta. Por ejemplo, el usuario escribe hola donde esperábamos un número.



Una excepción es un problema que Java puede detectar durante la ejecución.

La meta no es esconder los errores: es manejarlos de forma controlada para informar al usuario o tomar otra decisión.

3.1. El caso clásico: texto a entero

```
String texto = "25";  
int edad = Integer.parseInt(texto);
```

Si el texto fuera abc, Java lanzaría `NumberFormatException`.

```
String texto = "abc";  
int edad = Integer.parseInt(texto); // NumberFormatException
```

3.2. try y catch

`try` contiene el código que puede producir una excepción. `catch` indica qué hacer si ocurre.

```
try {  
    int edad = Integer.parseInt(txtEdad.getText());  
    System.out.println("Edad: " + edad);  
} catch (NumberFormatException e) {  
    JOptionPane.showMessageDialog(  
        this, "Debe escribir un número entero."  
    );  
}
```

Léelo así: “intenta; si ocurre `NumberFormatException`, captura el problema y muestra este mensaje”.

3.3. ¿Qué significa la variable e?

En catch (NumberFormatException e), e contiene información sobre la excepción. Durante la depuración puedes usar e.getMessage() o e.printStackTrace().

3.4. finally

```
try {
    System.out.println("Intentando...");
} catch (Exception e) {
    System.out.println("Ocurrió un problema.");
} finally {
    System.out.println("Esta parte se ejecuta al final.");
}
```

3.5. Capturar una excepción específica

Si sabes qué problema puede ocurrir, captura la excepción específica. NumberFormatException explica mejor el problema que capturar Exception indiscriminadamente.

3.6. throw

Puedes lanzar una excepción cuando una regla de tu aplicación determina que un dato es inválido.

```
int edad = Integer.parseInt(txtEdad.getText());

if (edad < 0) {
    throw new IllegalArgumentException(
        "La edad no puede ser negativa."
    );
}
```

3.7. Validación y excepción

Validación	Excepción
Comprueba reglas de los datos.	Representa una situación excepcional durante la ejecución.
Ejemplo: edad entre 0 y 120.	Ejemplo: parseInt recibe 'abc'.
Suele utilizar if.	Puede utilizar try/catch.

3.8. Ejemplo completo en NetBeans

Supón que tienes txtEdad y btnValidar.

```
private void btnValidarActionPerformed(java.awt.event.ActionEvent evt) {  
    try {  
        String textoEdad = txtEdad.getText().trim();  
  
        if (textoEdad.isEmpty()) {  
            JOptionPane.showMessageDialog(this, "Ingresa la edad.");  
            return;  
        }  
  
        int edad = Integer.parseInt(textoEdad);  
  
        if (edad < 0 || edad > 120) {  
            JOptionPane.showMessageDialog(  
                this, "La edad debe estar entre 0 y 120."  
            );  
            return;  
        }  
  
        JOptionPane.showMessageDialog(  
            this, "Edad válida: " + edad  
        );  
  
    } catch (NumberFormatException e) {  
        JOptionPane.showMessageDialog(  
            this, "La edad debe contener solamente números."  
        );  
    }  
}
```

Observa la diferencia: el if controla reglas normales; el try/catch controla el problema que puede ocurrir al convertir el texto.

4. JTable: ArrayList de objetos

Objetivo: guardar varios objetos creados por nosotros y mostrarlos en una tabla.

ArrayList<Persona>

[Persona 1] [Persona 2] [Persona 3]

JTable

Fila = objeto | Columna = atributo

La relación fundamental es: Objeto → ArrayList → Modelo de tabla → JTable.

El JTable muestra. El ArrayList mantiene la colección. El modelo conecta los datos con la interfaz.

4.1. Crear la clase Persona

En NetBeans: clic derecho sobre el paquete → New → Java Class → Persona.

```
public class Persona {  
  
    private String nombre;  
    private int edad;  
    private String ciudad;  
  
    public Persona(String nombre, int edad, String ciudad) {  
        this.nombre = nombre;  
        this.edad = edad;  
        this.ciudad = ciudad;  
    }  
  
    public String getNombre() {  
        return nombre;  
    }  
  
    public int getEdad() {  
        return edad;  
    }  
  
    public String getCiudad() {  
        return ciudad;  
    }  
}
```

Cada Persona es un objeto con atributos que contienen sus datos.

4.2. Crear el ArrayList

```
private ArrayList<Persona> listaPersonas = new ArrayList<>();  
  
import java.util.ArrayList;  
  
listaPersonas.add(  
    new Persona("Ana", 20, "Santa Marta")  
);
```

```
listaPersonas.add(  
    new Persona("Luis", 22, "Bogotá")  
);
```

4.3. Preparar el jTable en NetBeans

- 1 Arrastra un Table desde Palette al JFrame.
- 2 Cambia su variable a tblPersonas.
- 3 Crea un JButton llamado btnCargar si quieres probar la carga por separado.

4.4. DefaultTableModel

Para comenzar, DefaultTableModel es práctico porque permite agregar filas directamente.

```
import javax.swing.table.DefaultTableModel;

private DefaultTableModel modelo;

private void configurarTabla() {
    modelo = new DefaultTableModel(
        new Object[]{"Nombre", "Edad", "Ciudad"}, 0
    );

    tblPersonas.setModel(modelo);
}
```

Llama a configurarTabla() después de initComponents():

```
public FrmPersonas() {
    initComponents();
    configurarTabla();
}
```

4.5. Cargar el ArrayList en la tabla

```
private void cargarTabla() {

    modelo.setRowCount(0);

    for (Persona persona : listaPersonas) {
        modelo.addRow(new Object[]{
            persona.getNombre(),
            persona.getEdad(),
            persona.getCiudad()
        });
    }
}
```

setRowCount(0) evita duplicar filas cuando recargas la tabla.

4.6. ¿Qué hace el for?

```
for (Persona persona : listaPersonas) {  
    ...  
}
```

Se lee: “por cada objeto Persona de listaPersonas, ejecuta este bloque”.

Cada objeto aporta sus atributos para construir una fila.

```
Object[] fila = {  
    persona.getNombre(),  
    persona.getEdad(),  
    persona.getCiudad()  
};
```

4.7. Agregar desde JTextField y actualizar el JTable

Supón txtNombre, txtEdad, txtCiudad y btnAgregar. Aquí se combinan los tres temas.

```
private void btnAgregarActionPerformed(java.awt.event.ActionEvent evt) {  
  
    try {  
        String nombre = txtNombre.getText().trim();  
        String textoEdad = txtEdad.getText().trim();  
        String ciudad = txtCiudad.getText().trim();  
  
        if (nombre.isEmpty() || textoEdad.isEmpty() || ciudad.isEmpty()) {  
            JOptionPane.showMessageDialog(this, "Complete todos los campos.");  
            return;  
        }  
  
        int edad = Integer.parseInt(textoEdad);  
  
        if (edad < 0 || edad > 120) {  
            JOptionPane.showMessageDialog(  
                this, "La edad debe estar entre 0 y 120."  
            );  
            return;  
        }  
  
        Persona persona = new Persona(nombre, edad, ciudad);  
        listaPersonas.add(persona);  
        cargarTabla();  
  
    } catch (NumberFormatException e) {  
        JOptionPane.showMessageDialog(  
            this, "La edad debe ser un número."  
        );  
    }  
}
```

Aquí tenemos: excepción para la conversión, ArrayList para guardar y JTable para mostrar.

4.8. Leer la fila seleccionada

Una forma sencilla es usar el índice seleccionado y recuperar el objeto desde el ArrayList, siempre que el orden coincida.

```
int fila = tblPersonas.getSelectedRow();

if (fila == -1) {
    JOptionPane.showMessageDialog(
        this, "Seleccione una persona."
    );
    return;
}

Persona persona = listaPersonas.get(fila);

JOptionPane.showMessageDialog(
    this, "Nombre: " + persona.getNombre()
);
```

Si posteriormente ordenas o filtras el JTable, la fila visual puede no coincidir con el índice del ArrayList. En ese caso hay que convertir el índice de la vista al índice del modelo.

4.9. ¿Por qué separar ArrayList y JTable?

Elemento	Responsabilidad
Persona	Representa un registro individual.
ArrayList	Mantiene muchos registros en memoria.
DefaultTableModel	Conecta datos con filas y columnas.
JTable	Muestra los datos al usuario.

5. Proyecto integrador: Registro de Personas

Construiremos una aplicación pequeña que integre los tres temas.

5.1. Componentes

Componente	Variable	Uso
TextField	txtNombre	Nombre
TextField	txtEdad	Edad
TextField	txtCiudad	Ciudad
RadioButton	rbtnEstudiante	Tipo
RadioButton	rbtnDocente	Tipo
RadioButton	rbtnAdministrativo	Tipo
ButtonGroup	grupoTipo	Agrupar radios
Button	btnAgregar	Agregar
Table	tblPersonas	Mostrar registros

5.2. Clase Persona ampliada

```
public class Persona {  
  
    private String nombre;  
    private int edad;  
    private String ciudad;  
    private String tipo;  
  
    public Persona(String nombre, int edad,  
                   String ciudad, String tipo) {  
        this.nombre = nombre;  
        this.edad = edad;  
        this.ciudad = ciudad;  
        this.tipo = tipo;  
    }  
  
    public String getNombre() { return nombre; }  
    public int getEdad() { return edad; }  
    public String getCiudad() { return ciudad; }  
    public String getTipo() { return tipo; }  
}
```

5.3. Variables del formulario

```
private ArrayList<Persona> listaPersonas = new ArrayList<>();  
private DefaultTableModel modelo;
```

5.4. Configuración inicial

```
public FrmPersonas() {  
    initComponents();  
    configurarTabla();  
  
    grupoTipo.add(rbtnEstudiante);  
    grupoTipo.add(rbtnDocente);  
    grupoTipo.add(rbtnAdministrativo);  
}
```

Si NetBeans ya configuró el ButtonGroup visualmente, no vuelvas a agregar los radios por código. El ejemplo sirve para comprender la lógica equivalente.

5.5. Configurar la tabla

```
private void configurarTabla() {  
    modelo = new DefaultTableModel(  
        new Object[]{"Nombre", "Edad", "Ciudad", "Tipo"}, 0  
    );  
    tblPersonas.setModel(modelo);  
}
```

5.6. Obtener el tipo seleccionado

```
private String obtenerTipoSeleccionado() {  
  
    if (rbtnEstudiante.isSelected()) {  
        return "Estudiante";  
    }  
  
    if (rbtnDocente.isSelected()) {  
        return "Docente";  
    }  
  
    if (rbtnAdministrativo.isSelected()) {  
        return "Administrativo";  
    }  
  
    return null;  
}
```

Separar esta tarea en un método hace que el evento del botón sea más fácil de leer.

5.7. Cargar la tabla

```
private void cargarTabla() {

    modelo.setRowCount(0);

    for (Persona persona : listaPersonas) {
        modelo.addRow(new Object[]{
            persona.getNombre(),
            persona.getEdad(),
            persona.getCiudad(),
            persona.getTipo()
        });
    }
}
```

5.8. Evento Agregar

```
private void btnAgregarActionPerformed(java.awt.event.ActionEvent evt) {

    try {
        String nombre = txtNombre.getText().trim();
        String textoEdad = txtEdad.getText().trim();
        String ciudad = txtCiudad.getText().trim();

        if (nombre.isEmpty()
            || textoEdad.isEmpty()
            || ciudad.isEmpty()) {
            JOptionPane.showMessageDialog(
                this, "Complete todos los campos."
            );
            return;
        }

        String tipo = obtenerTipoSeleccionado();

        if (tipo == null) {
            JOptionPane.showMessageDialog(
                this, "Seleccione un tipo de usuario."
            );
            return;
        }

        int edad = Integer.parseInt(textoEdad);

        if (edad < 0 || edad > 120) {
            JOptionPane.showMessageDialog(
                this, "La edad debe estar entre 0 y 120."
            );
            return;
        }

        Persona persona = new Persona(
            nombre, edad, ciudad, tipo
        );

        listaPersonas.add(persona);
        cargarTabla();

    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(
            this, "La edad debe ser un número entero."
        );
    }
}
```

5.9. ¿Qué sucede al pulsar Agregar?

- 1 Se leen los textos.
- 2 Se verifica que no estén vacíos.
- 3 Se consulta el JRadioButton seleccionado.
- 4 Se intenta convertir la edad.
- 5 Si falla la conversión, catch muestra un mensaje.
- 6 Se crea un objeto Persona.
- 7 El objeto entra al ArrayList.
- 8 Se actualiza el JTable.

Cadena completa

Usuario → Formulario → Validación → Objeto → ArrayList → JTable

6. Errores frecuentes

Problema	Causa probable	Qué revisar
Varios radios seleccionados	No comparten ButtonGroup.	Revisa la propiedad buttonGroup.
NumberFormatException	Texto no numérico.	Usa try/catch y valida.
JTable vacío	No se cargó o el modelo no está asignado.	Revisa configurarTabla() y cargarTabla().
Filas duplicadas	Se recarga sin limpiar.	Usa setRowCount(0).
Cannot find symbol	Nombre incorrecto o falta import.	Lee exactamente el identificador.
NullPointerException	Una referencia es null.	Busca qué objeto no fue creado.

6.1. Técnica de depuración

- 1 Lee el mensaje de error.
- 2 Ubica la línea marcada por NetBeans.
- 3 Revisa nombres y tipos.
- 4 Pregunta si el objeto existe.
- 5 Corrige una sola cosa.
- 6 Ejecuta nuevamente.

Recuerda: un error no significa que no puedas programar. Es información sobre algo que debes corregir.

7. Ejercicios graduados

Trabaja de menor a mayor dificultad. Intenta explicar el ejercicio antes de mirar una solución.

Nivel 1 — ButtonGroup

- Crea Rojo, Verde y Azul; agrúpalos y muestra cuál fue seleccionado.
- Agrega una opción Ninguno y decide si debe pertenecer al mismo grupo.
- Configura una opción por defecto con `setSelected(true)`.

Nivel 2 — Excepciones

- Controla `NumberFormatException` en un campo de edad.
- Calcula una división y decide cómo manejar un divisor 0.
- Diferencia mensaje de campo vacío y mensaje de número inválido.

Nivel 3 — JTable + objetos

- Crea Producto con código, nombre y precio.
- Crea `ArrayList`.
- Muestra los productos en un `JTable`.
- Agrega productos desde `JTextField`.

Nivel 4 — Integración

- Crea Estudiante con nombre, edad, programa y jornada.
- Usa `JRadioButton` + `ButtonGroup` para jornada.
- Usa `try/catch` para edad.
- Guarda en `ArrayList` y muestra en `JTable`.

Reto final

Agrega Editar y Eliminar. Al seleccionar una fila, carga sus datos en los controles. Para eliminar, pide confirmación y después actualiza el `ArrayList` y el `JTable`.

8. Preguntas de comprensión

Tema	Debes poder responder
ButtonGroup	¿Por qué el grupo no aparece en la ventana?
JRadioButton	¿Qué devuelve isSelected()?
try/catch	¿Qué código debe estar dentro de try?
NumberFormatException	¿Cuándo aparece normalmente?
ArrayList	¿Qué contiene cada elemento?
DefaultTableModel	¿Por qué usamos setRowCount(0)?
JTable	¿Debe ser el lugar donde viven nuestros objetos?

Pregunta de sustentación

“Explique el recorrido de un dato desde que el usuario lo escribe hasta que aparece en el JTable.”

Respuesta esperada: el usuario escribe; el programa valida; convierte los tipos; crea un objeto; guarda el objeto en ArrayList; recorre la lista y coloca sus atributos en el DefaultTableModel; el JTable muestra ese modelo.

9. Hoja rápida de consulta

ButtonGroup + JRadioButton

```
if (rbtnOpcion.isSelected()) {  
    ...  
}  
  
ButtonGroup grupo = new ButtonGroup();  
grupo.add(rbtnA);  
grupo.add(rbtnB);  
grupo.add(rbtnC);
```

Excepciones

```
try {  
    int numero = Integer.parseInt(txtNumero.getText());  
} catch (NumberFormatException e) {  
    JOptionPane.showMessageDialog(  
        this, "Número inválido"  
    );  
}
```

ArrayList de objetos

```
private ArrayList<Persona> lista = new ArrayList<>();  
  
lista.add(new Persona("Ana", 20, "Santa Marta"));  
  
for (Persona p : lista) {  
    System.out.println(p.getNombre());  
}
```

JTable + DefaultTableModel

```
modelo = new DefaultTableModel(  
    new Object[]{"Nombre", "Edad"}, 0  
);  
  
tblPersonas.setModel(modelo);  
modelo.setRowCount(0);  
  
for (Persona p : lista) {  
    modelo.addRow(new Object[]{  
        p.getNombre(), p.getEdad()  
    });  
}
```

10. Lista de comprobación

- Los JRadioButton exclusivos comparten ButtonGroup.
- Los nombres de componentes coinciden con el código.
- Las conversiones de texto a número están controladas.
- Las validaciones están separadas del manejo de excepciones.
- La clase de objeto tiene atributos y getters necesarios.
- El ArrayList utiliza el tipo de objeto correcto.
- El JTable tiene un modelo configurado.
- La tabla se limpia antes de recargarse.
- Probaste datos correctos e incorrectos.
- Puedes explicar cada bloque principal con tus propias palabras.

Cierre

Los tres temas se conectan: JRadioButton + ButtonGroup capturan una decisión; las excepciones permiten responder de forma controlada a problemas; y ArrayList + JTable separan los datos de su representación visual.

La meta no es memorizar código. La meta es reconocer el recorrido: entrada → validación → objeto → colección → modelo → interfaz.